

Negotex — Unique Selling Points

Internal reference for positioning, marketing, sales conversations, and content creation.

The one-line pitch

Negotex is the only BPMN-compatible workflow engine that compiles process definitions directly to a Kafka message topology — eliminating the central orchestrator that makes every other engine a bottleneck and a single point of failure.

Core differentiators

1. No orchestrator — horizontal scaling by design

Every BPMN engine on the market (Camunda, Temporal, Conductor, Flowable) shares the same architectural pattern: a central coordinator polls a database and dispatches tasks. That coordinator is both the throughput ceiling and the single point of failure. You can scale everything around it; you cannot scale it.

Negotex has no coordinator. A process definition compiles to a Kafka topology. Each node is a stateless stream processor in a consumer group. Adding replicas absorbs load without any configuration change. Throughput scales with Kafka partition count and processor count — not with the capacity of a shared dispatcher.

The claim: 10x+ throughput vs. polling-based engines on equivalent hardware, with no architectural ceiling.

Who cares: Platform engineers running high-volume processes. Financial services with millions of daily transactions. Insurance claim processing at scale.

2. Tamper-evident audit trail — no competitor offers this natively

Every envelope transition is cryptographically chained:

```
EnvelopeHash = SHA-256(previousHash || executionResult || timestamp || handlerVersion)
```

The chain is verifiable on-demand or continuously by an Audit Verifier consumer. Any deletion, reordering, or modification of audit records breaks the chain at the tampered point.

Neither Camunda nor Temporal provide native cryptographic chaining over execution history. Blockchain-based approaches require external infrastructure and governance overhead Negotex doesn't need.

Who cares: Compliance officers in banking, insurance, and healthcare. Regulators requiring Basel III, SOX, or GDPR audit trail integrity. Any organisation that needs to prove process execution happened exactly as recorded.

3. BPMN as a first-class citizen — not a translation layer

Temporal is code-first — BPMN is an afterthought or unsupported. Camunda supports BPMN but its execution model diverges from the spec in ways that matter to regulatory environments where BPMN is a compliance artefact, not just a design tool.

Negotex maps BPMN concepts directly to execution primitives with no translation layer:

BPMN	Negotex
Sequence Flow	Kafka topic
Service Task	Map processor
Parallel Gateway	Fork / Join
Exclusive Gateway	Choice / Merge
Inclusive Gateway	Filter
Intermediate Event	Wait
Start / End Event	Trigger / Terminate

Who cares: Regulated industries where BPMN process definitions are submitted to regulators or auditors. Teams that model in BPMN and want the model to be the truth, not a diagram that gets thrown away after implementation.

4. Handlers as pure functions — zero infrastructure coupling

A Negotex handler is `Input → Output`. No Kafka client. No database connection. No framework imports. The handler has no knowledge of the topology it runs in.

```
public class CreditCheckHandler implements TaskHandler<LoanApplication, CreditResult> {  
    public CreditResult handle(LoanApplication application) {  
        // pure business logic – unit testable without any infrastructure  
    }  
}
```

This is not just developer convenience. It is the foundation for Consistency Contracts: a handler that has no side effects is provably deterministic, and provably deterministic handlers enable Audit Certification Mode — where every process instance is fully reproducible from the audit trail.

Who cares: Development teams who have suffered through testing Camunda delegates that require a full Spring context. Architects who want a clean separation between business logic and infrastructure.

5. Execution contracts — security posture declared at deployment

Every node declares what it is allowed to do. Every handler declares what it needs. The process compiler validates at deployment time that handler capabilities are a subset of node permissions. Deployment fails if any handler exceeds its policy.

```
// Node governance policy
{ "allowedExternalCalls": ["payment-gateway"], "maxExecutionTime": "2s" }

// Handler capability declaration
{ "allowedOutboundServices": ["payment-gateway"], "memoryLimit": "64MB" }
```

No other workflow engine offers deployment-time capability validation. In most engines, a handler that calls an external service it shouldn't is discovered at runtime, in production.

Who cares: Security teams in regulated environments. Compliance officers who need to demonstrate that process execution is bounded and auditable before it happens.

6. Zero-downtime versioning — structurally guaranteed

Version isolation is not a configuration option — it is structural. Each process version gets its own isolated Kafka topic namespace. In-flight instances on v1.2.0 complete on v1.2.0 topics while new instances start on v1.3.0 topics. No coordinated cutover. No migration scripts. No envelopes misrouted between versions.

Who cares: Platform engineers operating long-running processes (loan approvals, insurance claims, regulatory reviews) that cannot be interrupted for deployments.

7. Multilanguage — first-class, not External Task

Temporal and Conductor support multiple languages via polling workers — a second-class pattern where non-primary languages poll a task queue managed by the main runtime. This adds latency and creates an architectural citizen hierarchy.

In Negotex, Java, F#, and Rust are all first-class. Each language kit communicates directly with Kafka. A risk model written in F# participates in the same process topology as a Java service task with no intermediary.

F# is prioritised for Phase 2 precisely because financial services uses it extensively for quantitative modelling and risk calculations — the domain where Negotex's compliance features matter most.

Who cares: Financial services teams with F# quant models that need to participate in regulated workflows without being wrapped in a Java polling worker.

8. Open-core with a complete community edition

The community edition is not a teaser. Full workflow execution, all nine primitives, hash chains, execution contracts, consistency contracts, and the OSS console are available under Apache 2.0. Any team can run Negotex in production without a commercial licence.

The Enterprise Control Plane adds operational capabilities — multi-cluster management, blue/green deployments, compliance reporting, process replay debugging — for organisations that outgrow the community edition. It never executes workflow logic, maintaining a clean IP boundary.

Pricing is per-cluster, not per-execution. No metering surprises as process volume grows.

Who cares: Teams evaluating open-source workflow engines who have been burned by Camunda's shift to source-available licensing. CTOs who need a credible upgrade path without a fork risk.

9. Infrastructure-agnostic by design.

Swap Kafka for Google Pub/Sub. Swap TimescaleDB for Oracle. One Maven dependency, one Spring bean — no process definition changes, no migration scripts, no vendor lock-in.

Competitive summary

	Negotex	Camunda 8	Temporal	Conductor
No central orchestrator	✓	✗	✗	✗
Native BPMN execution	✓	✓	✗	Partial
Cryptographic audit trail	✓	✗	✗	✗
Deployment-time capability validation	✓	✗	✗	✗
Handlers as pure functions	✓	✗	Partial	✗
First-class multilanguage (no polling)	✓	✗	✗	✗
Apache 2.0 complete edition	✓	✗ (SSPL)	✓	✓
Per-cluster pricing	✓	✗ (per-core)	✗ (consumption)	✗

Target positioning statement

For compliance-heavy industries that require BPMN-compatible process execution with a verifiable audit trail, Negotex is the only workflow engine that eliminates the central orchestrator bottleneck while delivering cryptographic tamper evidence, deployment-time security contracts, and deterministic process reproducibility — capabilities no competing engine offers natively.